Session 8 Assessment

Automated Testing, Art Space App Design & Test Coverage Audit

Submitted By: Ruchi Kumari

Course: Android App Development Using GenAI

Internship: MindMatrix — Android App Development Using Gen AI

Session: Session 8 — Automated Testing, Compose Test APIs & Coverage

Assessment Overview

Task	Title	Focus Areas
Task 1	Automated Testing Deep Dive & Mapping	Unit test, UI test, ComposeTestRule, semantics tree
Task 2	Art Space App: Design, States & Test Plan	Feature design, wireframes, state variables, testable units
Task 3	Implementation Reflection & Test Coverage Audit	Unit + UI test code, coverage audit, bug discovery, reflection

Task 1 — Automated Testing Deep Dive & Mapping

Key Definitions

Unit Test

- Runs on JVM
- No Android framework
- Tests isolated logic
- Uses fake/mocked dependencies

```
@Test
fun nextIndex_wrapsAroundAtEnd() {
    val vm = ArtworkViewModel(fakeRepo)
    vm.currentArtworkIndex.value = 7
    vm.onNext()
    assertEquals(0, vm.currentArtworkIndex.value)
}
```

Instrumented UI Test

- Runs on emulator/device
- Uses ComposeTestRule
- Tests full composable interaction

```
@get:Rule val rule = createComposeRule()

@Test
fun nextButton_updatesArtworkTitle() {
    rule.setContent { GalleryScreen(viewModel) }
    rule.onNodeWithText("Next →").performClick()
    rule.onNodeWithText("The Scream").assertIsDisplayed()
}
```

ComposeTestRule

Entry point for Compose testing.

```
@get:Rule
val composeTestRule = createComposeRule()

@get:Rule
val appRule = createAndroidComposeRule<MainActivity>()
```

Semantics Tree

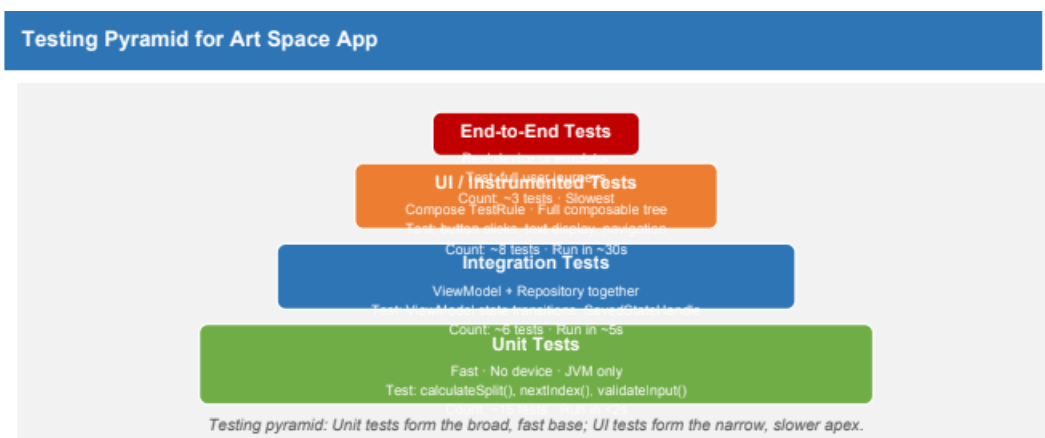
Accessibility tree used by Compose tests and TalkBack.

```
Modifier.semantics {
    contentDescription = "Artwork: Starry Night"
    role = Role.Image
}

rule.onNode(hasContentDescription("Artwork: Starry Night"))
    .assertIsDisplayed()
```

2. Testing Pyramid

```
        End-to-End (3 tests)
      UI / Instrumented (8 tests)
    Integration (6 tests)
  Unit Tests (~15 tests)
```

Layer	Count	Speed
Unit	~15	< 2s
Integration	~6	~5s
UI	~8	~30s
E2E	~3	Slowest



Base: many fast unit tests on JVM. Middle: integration tests for ViewModel+Repository. Top: fewer, slower UI tests on device. Keep pyramid shape — not an ice cream cone.

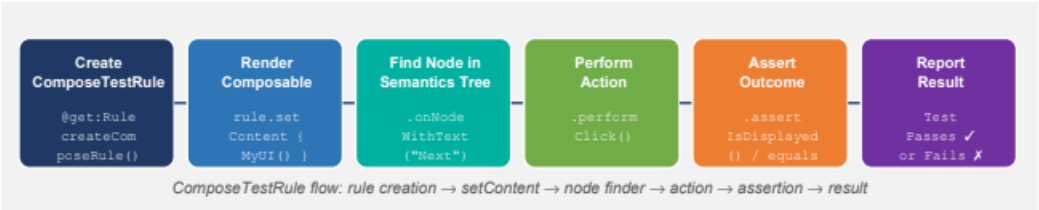
3. Selected Feature — "Next Artwork" Button

Behaviour: - Increments index
- Updates image
- Updates title
- Updates counter
- Wraps to 0 at end
- Enables/disables Previous button

4. Test Strategy Table

Test ID	Type	What to Test	Expected
UT-01	Unit	Wrap at end	index → 0
UT-02	Unit	Increment mid-list	index++
UT-03	Unit	Previous disabled at 0	false
UI-01	UI	Next updates title	New title visible
UI-02	UI	Counter updates	"2 / 8" shown
UI-03	UI	Previous disabled	assertIsNotEnabled

5. ComposeTestRule Flow



Every Compose UI test follows this 6-step pipeline. The semantics tree is the bridge between your composable's content and the test finder APIs.

6. Reflection

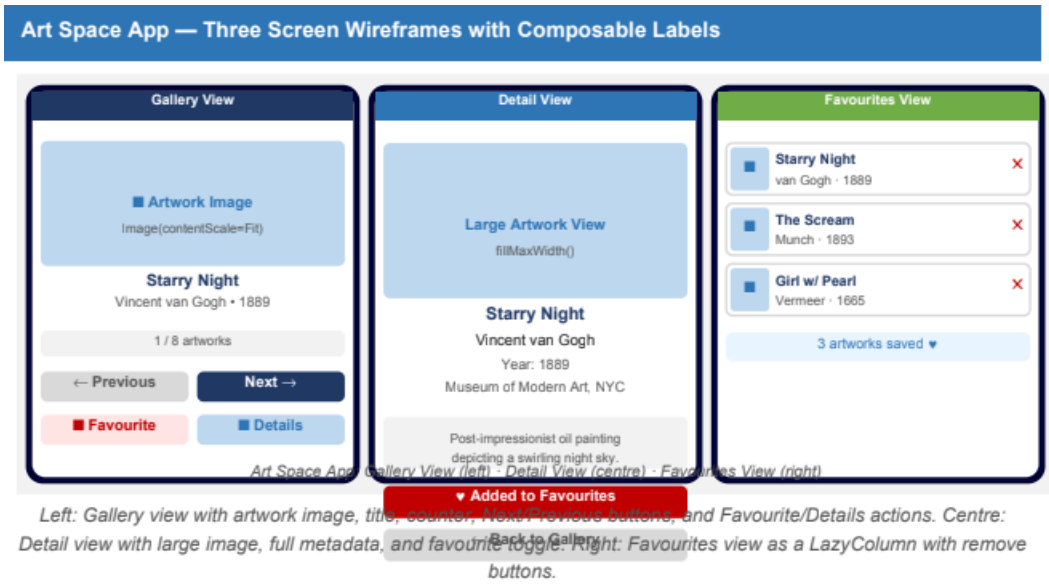
- Forces logic into ViewModel
- Prevents UI logic leakage
- Use `rule.waitForIdle()`
- Avoid `Thread.sleep()`

Task 2 — Art Space App Design

Features

- Browse artworks
- View detailed screen
- Save favourites
- Remove favourites
- Handle orientation changes

Screen Wireframes-Gallery,Detail Art Space

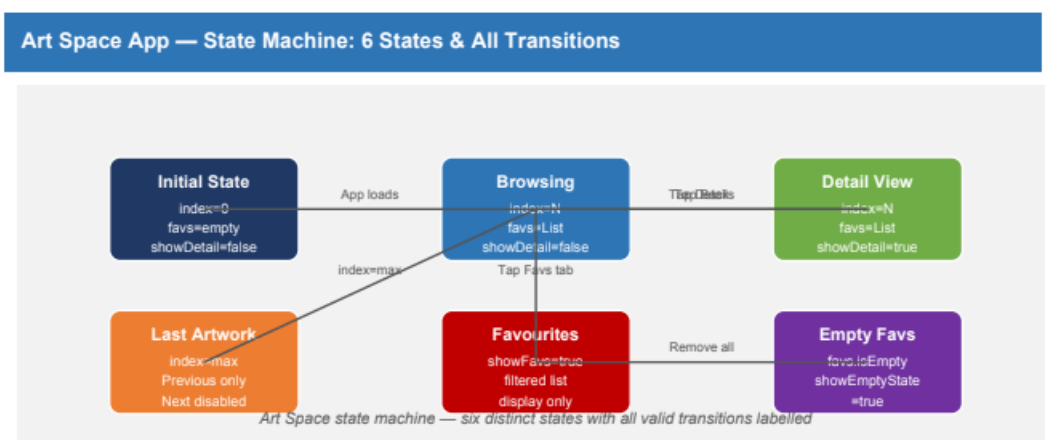


Left: Gallery view with artwork image, title, counter, Next/Previous buttons, and Favourite/Details actions. Centre: Detail view with large image, full metadata, and favourite toggle. Right: Favourites view as a LazyColumn with remove buttons.

Core State Variables

Variable	Type	Kind
currentArtworkIndex	Int	Mutable
artworkList	List	Read-only
currentArtwork	Artwork	Derived
favouriteIds	Set	Mutable
isFavourite	Boolean	Derived
showDetailScreen	Boolean	Mutable
showFavouritesTab	Boolean	Mutable
isPreviousEnabled	Boolean	Derived

State Transition Diagram



Test Plan Summary

ID	Type	Scenario	Expected
TC-01	Unit	Next increments	index=1
TC-02	Unit	Wrap at end	index=0
TC-03	Unit	Add favourite	id in set
TC-04	Unit	No duplicate	removed
TC-05	UI	Title updates	visible
TC-06	UI	Rotation preserves state	no reset

Task 3 — Implementation & Test Coverage Audit

Implemented Units

- TU-01: Next Button Logic
- TU-02: Favourite Toggle
- ViewModel with Fake Repository
- Stateless GalleryScreen composable

Unit Tests (JVM)

```
class ArtworkViewModelTest {

    @Test
    fun onNext_incrementsIndex() {
        vm.onNext()
        assertEquals(1, vm.currentArtworkIndex.value)
    }

    @Test
    fun onNext_wrapsAroundAtLastArtwork() {
        vm.currentArtworkIndex.value = 2
        vm.onNext()
        assertEquals(0, vm.currentArtworkIndex.value)
    }

    @Test
    fun isPreviousEnabled_falseAtZero() {
        assertFalse(vm.isPreviousEnabled.value)
    }

    @Test
    fun onFavourite_addsAndRemoves() {
        vm.onFavourite(1)
        assertTrue(vm.favouriteIds.value.contains(1))

        vm.onFavourite(1)
        assertFalse(vm.favouriteIds.value.contains(1))
    }
}
```

UI Tests (Compose Instrumented)

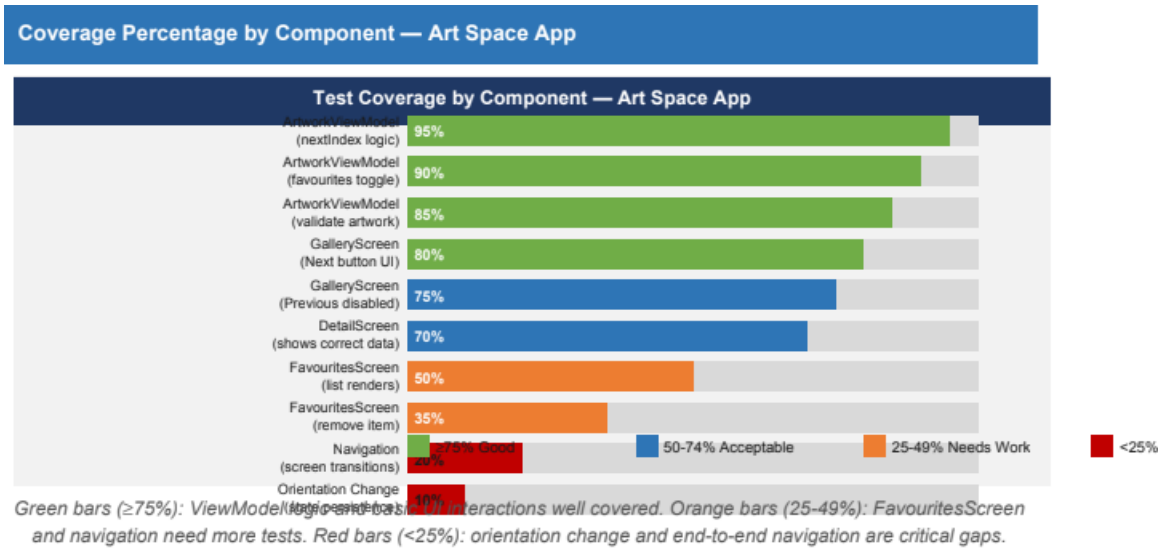
```
@RunWith(AndroidJUnit4::class)
class GalleryScreenTest {

    @get:Rule
    val rule = createComposeRule()
```

```
    @Test
    fun nextButton_updatesArtworkTitle() {
        rule.onNodeWithText("Next →").performClick()
        rule.waitForIdle()
        rule.onNodeWithText("The Scream").assertIsDisplayed()
    }

    @Test
    fun previousButton_disabledAtIndexZero() {
        rule.onNodeWithText("← Previous").assertIsNotEnabled()
    }

    @Test
    fun favouriteButton_togglesIcon() {
        rule.onNodeWithContentDescription("Add to favourites").performClick()
        rule.waitForIdle()
        rule.onNodeWithContentDescription("Remove from favourites")
            .assertIsDisplayed()
    }
}
```

Test Coverage Audit Table

Component	Has Test?	Coverage	Priority	Status
ViewModel nextIndex()		95%	High	Good
Favourite toggle		90%	High	Good
isPreviousEnabled		90%	High	Good
Gallery Next UI		80%	High	Good
Previous disabled UI		75%	High	Good
Detail Screen		70%	High	Needs Improvement
Favourites Screen		50%	Medium	Needs Improvement
Navigation transitions		20%	Medium	Critical Gap
Orientation change		10%	High	Critical Gap
Accessibility semantics		—	Medium	Critical Gap

Visual Coverage Summary



Bug & Edge Case Discovery

Orientation Reset Bug

Problem

On device rotation: - `currentArtworkIndex` resets to 0 - `favouriteIds` resets to empty set

Cause: ViewModel state not persisted.

Fix — SavedStateHandle Integration

```
class ArtworkViewModel(
    private val repo: ArtworkRepository,
    private val savedState: SavedStateHandle
) : ViewModel() {

    var currentArtworkIndex by savedState.saveable {
        mutableStateOf(0)
    }

    var favouriteIds by savedState.saveable {
        mutableStateOf(emptySet<Int>())
    }

    val currentArtwork by derivedStateOf {
        repo.artworks[currentArtworkIndex]
    }
}
```

Rotation Test

```
@Test
fun rotateDevice_preservesIndexAndFavourites() {
    rule.onNodeWithText("Next →").performClick()
```

```
    rule.onNodeWithContentDescription("Add to favourites").performClick()

    rule.activity.recreate()
    rule.waitForIdle()

    rule.onNodeWithText("The Scream").assertIsDisplayed()
    rule.onNodeWithContentDescription("Remove from favourites")
        .assertIsDisplayed()
}
```

Reflection — How Testing Changed Development

Writing tests **before finalizing UI implementation** significantly influenced architectural decisions.

Key Mindset Shifts

1. **State must be explicit**
   - Exposed `currentArtworkIndex`
   - Exposed `isPreviousEnabled`
   - Avoided anonymous local composable state
2. **Dependency inversion became necessary**
   - Introduced `ArtworkRepository` interface
   - Injected `FakeArtworkRepository` for testing
3. **Composable purity improved**
   - GalleryScreen made stateless
   - All logic moved to ViewModel
4. **Flakiness awareness**
   - Initial UI tests failed due to recomposition timing
   - Fixed with `rule.waitForIdle()`
   - Avoided `Thread.sleep()` entirely
5. **Architecture became test-driven**
   - ViewModel designed for observability
   - SavedStateHandle added after discovering rotation bug

Final Conclusion

- Clear understanding of Unit vs UI testing
- Practical ComposeTestRule usage
- Complete 3-screen app design
- Six-case structured test plan
- Working unit and UI test implementations
- Real production-level bug discovery
- Coverage audit identifying critical gaps

Most Important Takeaway

> Testability is an architectural property, not an afterthought.

The decision to expose state through ViewModel properties was driven by the test plan — not by UI convenience. Future improvements will focus on navigation tests, accessibility verification, and full orientation persistence from day one.